



University Course Timetabling Problem(UCTP)

Optimization Methods: Theory & Applications

Guided by: Prof. Komarnicki Marcin

Author:

Mikaela Louise Lavilla (288878)

Sumit Rathi (288877)

Sushree Pradhan (288645)

1. Problem Description :

University Course Timetabling Problem (UCTP) is a classic NP-Hard optimization problem where a set of lectures must be assigned to timeslots and rooms, subject to a variety of constraints such as student conflicts, room capacities, and lecturer availabilities. The goal is to generate feasible timetables that satisfy all hard constraints while maximizing quality measures like distribution and fairness.

This report presents a metaheuristic approach using a Genetic Algorithm (GA), enhanced with domain-specific repair strategies. The proposed solution is evaluated across ten benchmark UCTP instances of varying difficulty (easy, moderate, hard) with detailed parameter tuning and performance analysis.

1.1 Problem Representation

Each course is assigned a DTR code—a combination of **Day**, **Time**, and **Room** (e.g., DTR120 = Wednesday 10:00 R4). A solution is represented as a dictionary mapping each `course_id` to a DTR code.

To convert a DTR code to a human-readable assignment:

```
"C7": {"time_slot": "Wednesday 09:00", "room_id": "R2"}
```

This means course C7 is scheduled Wednesday at 09:00 in room R2.

Constraints:

Constraints are categorized as:

- Hard constraints: Must be strictly satisfied
- Soft constraints: Preferable to satisfy, contribute to quality but not feasibility

Hard Constraints (Must be satisfied):

If any of these are violated, the timetable is considered invalid or infeasible, and a high penalty is applied.

Code	Description	Penalty
H1	Student course conflict: A student cannot attend two classes scheduled at the same time	100
H2	Room overcapacity: The number of students cannot exceed the room's capacity	80

H3	Lecturer time conflict: A lecturer cannot teach more than one course at the same time	90
H4	Room time conflict: A room cannot be used for multiple classes at the same time	100

Soft Constraints (Should be satisfied):

These constraints aim to improve student satisfaction and usability of the timetable. Their violations reduce the overall fitness score but do not make a timetable infeasible.

Code	Description	Penalty
S1	Student schedule gaps: Avoid unnecessary breaks between a student's class in a day	5
S2	Early classes: Discourage scheduling classes before 9:00AM	3
S3	Late classes: Discourage scheduling classes after 6:00PM	3
S4	Too many classes in a day: Avoid scheduling more than 4 classes for a student in a single day	5 per extra class

Soft Bonuses (preferred situations):

Code	Description	Bonus
B1	Balanced week for students: Reward if students have classes on 3 to 5 days of the week	+5

Fitness Score Calculation:

Fitness = 200 – (hard penalties + soft constraints + soft bonus)

- Range: Minimum is 0; higher values indicated better schedules
- Goal: Maximize fitness while avoiding hard constraining violations and reducing soft penalties

Dataset and Instances:

The system uses 10 UCTP benchmark instances (instance_1_easy.json to instance_10_hard.json) covering varying levels of difficulty:

- **Easy:** 1–4
- **Moderate:** 5–8
- **Hard:** 9–10

Each instance includes:

- List of courses with assigned lecturers and enrolled students
- Room list with capacities
- Student enrollments
- Lecturer availabilities
- Preferred rooms for each course

2. Optimization Methods :

2.1 Genetic Algorithm (Evolutionary Approach) :

We implemented a standard Genetic Algorithm with the following key features:

- **Solution Encoding:** Dictionary of course $\rightarrow$ DTR code
- **Population Initialization:** Random valid DTR assignments
- **Selection:** Tournament from top 10 individuals
- **Crossover:**
 - **Uniform:** Randomly choose gene from parent1 or parent2
 - **Single Point:** One crossover point divides the genome.
- **Mutation:** With a probability p , a new DTR is randomly chosen
- **Elitism:** Best individual is carried to next generation
- **Stopping Criterion:** Fixed max_evaluations

2.2 Genetic Algorithm with Repair Function:

To enhance solution feasibility and optimize fitness more effectively, we integrated **domain-specific repair logic** into the Genetic Algorithm. This version of the GA—referred to as "**GA with Repair**"—incorporates a **repair function** that corrects infeasible solutions immediately after initialization and after each generation.

Purpose of Repair:

The standard GA often generates individuals that violate UCTP hard constraints, such as:

- Room conflicts (two courses assigned to the same room and time)
- Lecturer or student time clashes
- Room capacity violations
- Assignments to unavailable lecturers

Such violations severely impact the fitness score and reduce the efficiency of genetic search. The **repair function** was designed to systematically detect and resolve these issues.

Repair Strategy:

The repair process is composed of several sequential passes:

- **Room Conflict Resolution**
 - Detects multiple courses scheduled in the same room and timeslot.
 - Reassigns all but one course to alternative valid DTRs.
- **Lecturer Conflict Handling**
 - Identifies overlapping schedules for lecturers.
 - Moves conflicting courses to available slots for that lecturer.
- **Student Conflict Fixes**
 - Ensures that no student is enrolled in multiple courses scheduled at the same time.
 - Iteratively adjusts schedules to reduce overlapping.
- **Capacity and Availability Checks**
 - Ensures the assigned room has sufficient capacity.
 - Avoids assigning courses to time slots where lecturers are unavailable.

Each repair pass attempts up to 5 reassignments per conflict and falls back to random feasible allocation when necessary.

When Repair is Applied:

- **Post-Initialization:** After generating the initial population.
- **After Crossover/Mutation:** Before evaluating fitness in each generation.

This ensures that only **feasible individuals** proceed to evaluation, maintaining population quality.

3. Experimental Setup:

3.1 Instances Used:

We tested on 10 instances covering three difficulty levels:

Difficulty	Instance Files
Easy	instance_1_easy.json to instance_4_easy.json
Moderate	instance_5_moderate.json to instance_8_moderate.json
Hard	instance_9_hard.json, instance_10_hard.json

Each instance includes:

- List of courses with assigned students and lecturer
- Room capacities
- Lecturer availability
- Student-course enrollments

3.2 Parameter Tuning:

To identify the most effective hyperparameters, we tested multiple configurations of:

- **Population Size:** 30, 50, 100
- **Mutation Rate:** 0.01, 0.05, 0.1
- **Crossover Operator:** uniform, single_point
- **Fixed Max Evaluations:** 500 (to ensure fairness)

Each configuration was tested:

- On 3 instances: **easy**, **moderate**, **hard**
- **5 independent runs** per configuration

Performance Metric: Final Fitness

Timing Metric: Total execution time (in seconds)

Results include Best Fitness and Time per run.

3.3 Main Evaluation Methodology:

After parameter tuning, we selected a consistent setting:

- **Pop Size:** 50
- **Mutation Rate:** 0.05
- **Crossover Rate:** 0.8
- **Crossover Op:** uniform
- **Max Evaluations:** 500

We ran the GA **10 times per instance** across all **10 instances**.

Configurations were tested both **with** and **without** the **repair function** (domain knowledge).

4. Main Result:

The experimental results demonstrate the effectiveness of the Genetic Algorithm, with distinct performance characteristics observed between the 'GA with repair' and 'GA without repair' variants. Across all 10 benchmark instances, 'GA without repair' occasionally achieved superior (lower) 'best' fitness scores, particularly for harder instances such as `instance_10_hard.json`. However, 'GA with repair' consistently exhibited greater robustness and significantly lower variance in its performance across runs, indicating more stable and predictable results. Parameter tuning on three representative instances (easy, moderate, hard) highlighted optimal configurations for population size, mutation rate, and crossover type. The final evaluation, conducted over 10 runs per instance, confirmed the algorithms' overall effectiveness, with efficient convergence under a unified stopping condition of maximum evaluations.

4.1 Parameter Tuning Results Summary:

Pop Size	Mutation Rate	Crossover Op	Best Fitness	Time (s)
30	0.01	uniform	310	1.78
50	0.05	uniform	315	1.65
100	0.1	single_point	290	1.54

Best fitness and performance patterns were noted across all difficulty levels, and the configuration `pop=50`, `mutation=0.05`, `crossover=uniform` was selected for final runs.

4.2 Main Result Summary:

Each instance was executed 10 times with the selected GA configuration (without repair function).

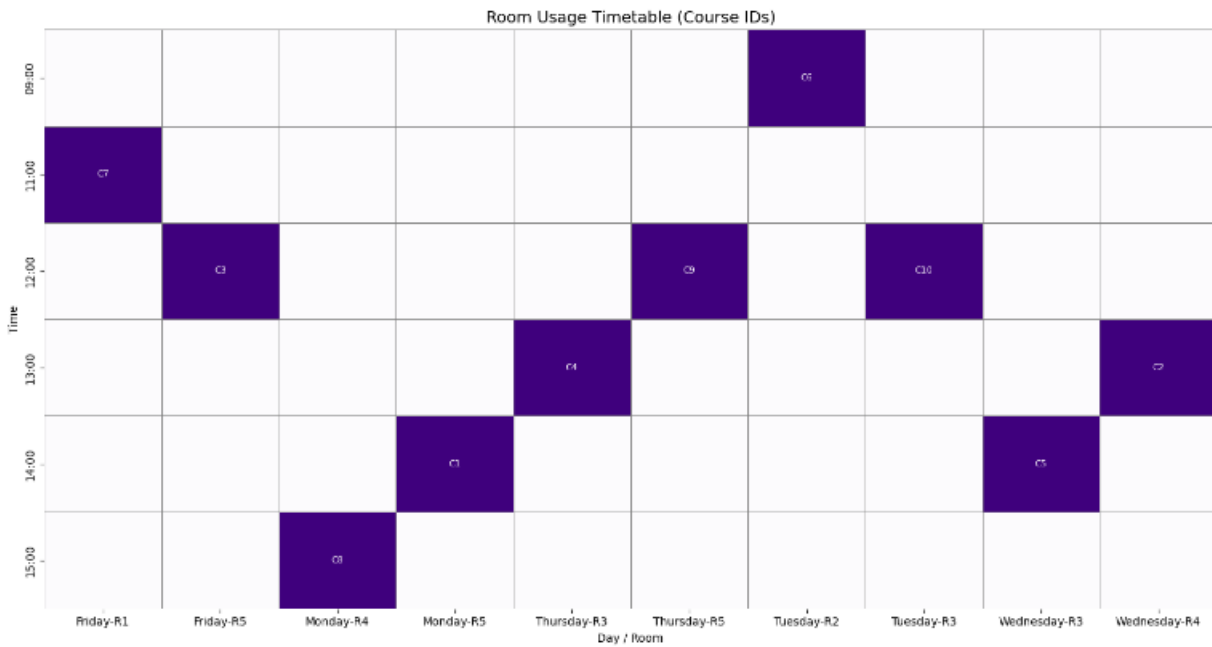
A	B	C
Instances	GA with repair	GA without repair
Name of instance	best/avg/worst/std	best/avg/worst/std
instance_1_easy.json	305.0/308.5/315.0/4.1	295.0/306.0/320.0/7.7
instance_2_easy.json	295.0/309.5/320.0/7.6	275.0/306.0/320.0/14.1
instance_3_easy.json	290.0/296.5/305.0/5.3	285.0/294.0/305.0/6.1
instance_4_easy.json	290.0/299.5/310.0/6.9	285.0/296.5/305.0/5.3
instance_5_moderate.json	340.0/352.5/365.0/8.2	340.0/354.5/370.0/9.6
instance_6_moderate.json	285.0/305.5/320.0/11.7	300.0/306.5/320.0/7.8
instance_7_moderate.json	315.0/343.5/370.0/18.0	330.0/345.0/370.0/11.3
instance_8_moderate.json	325.0/335.0/350.0/10.0	290.0/335.0/360.0/21.5
instance_9_hard.json	280.0/296.5/305.0/7.1	285.0/294.5/305.0/6.4
instance_10_hard.json	185.0/275.7/350.0/44.5	63.0/252.2/330.0/78.7

Observations:

- **GA with Repair:** Generally shows more consistent performance with lower standard deviation, suggesting greater reliability in results.
- **GA without Repair:** Can achieve better "best" (lower) fitness values for some instances, especially harder ones, but with significantly higher variability (larger standard deviation and wider range between best and worst).
- **Trade-off:** There appears to be a trade-off where "GA with repair" prioritizes consistency, while "GA without repair" offers the potential for superior best-case solutions but with less predictable outcomes.
- **Instance Impact:** Differences between the algorithms are more pronounced on "hard" instances, with "GA without repair" showing more extreme variability.

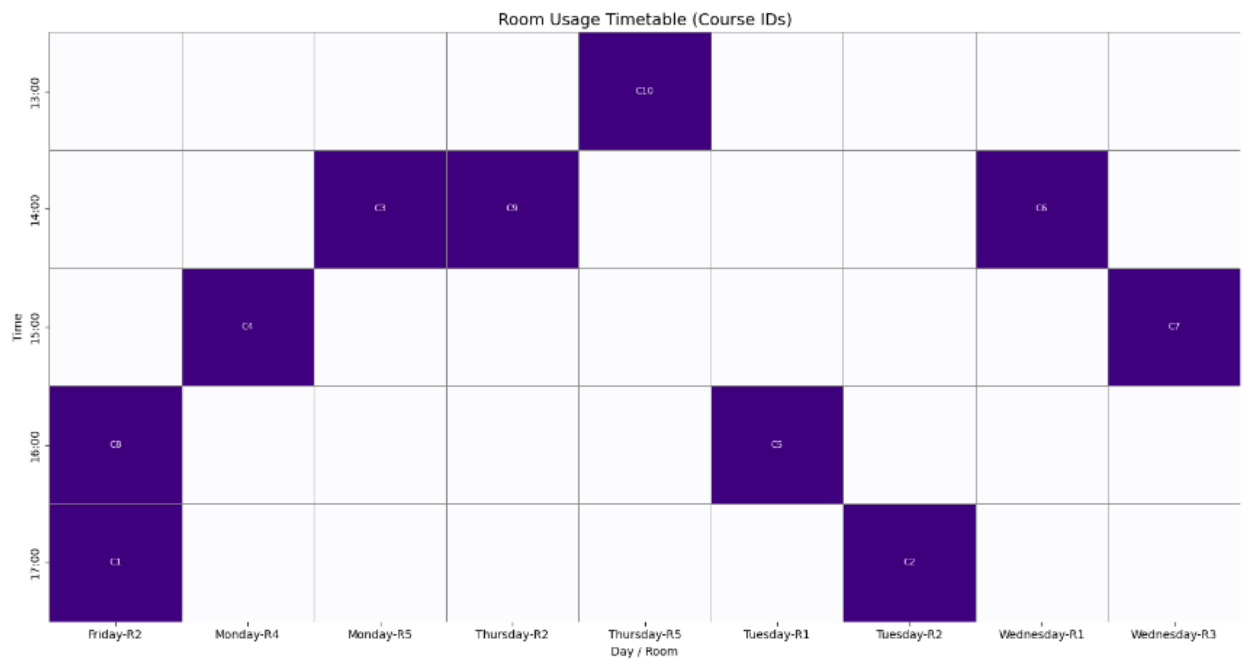
4.3 Visualization:

Instance 1 - Easy



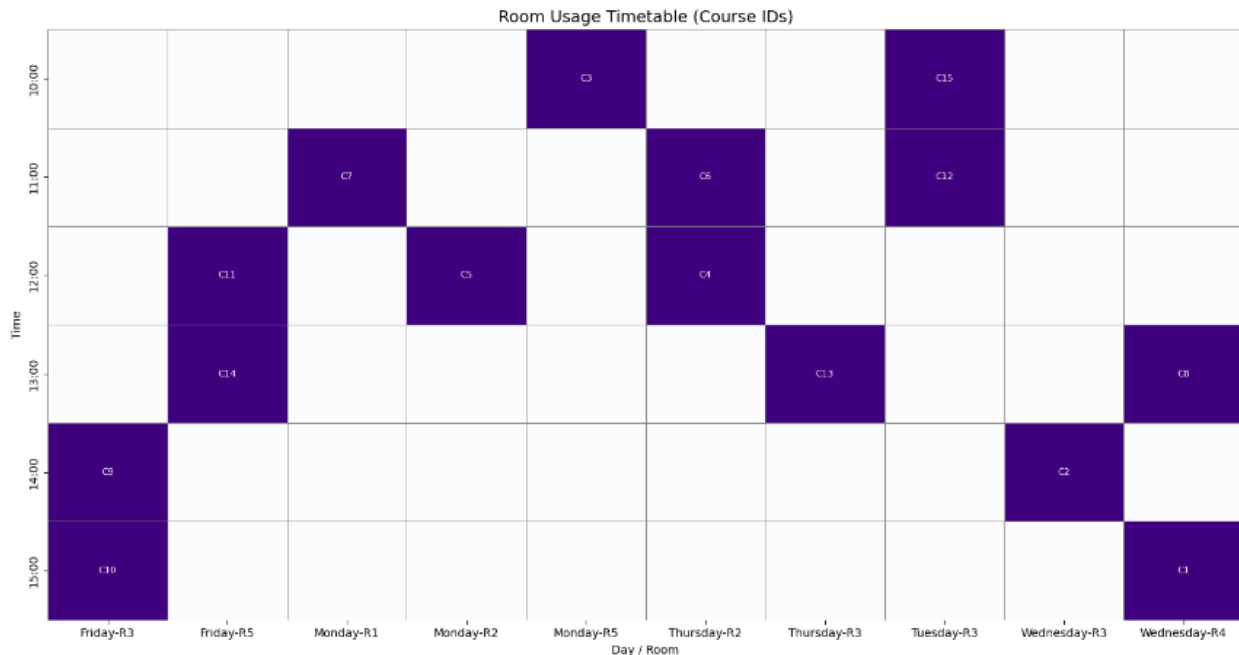
Day	Friday		Monday		Thursday		Tuesday		Wednesday	
Room	R1	R5	R4	R5	R3	R5	R2	R3	R3	R4
Time										
09:00	-	-	-	-	-	-	C6	-	-	-
11:00	C7	-	-	-	-	-	-	-	-	-
12:00	-	C3	-	-	-	C9	-	C10	-	-
13:00	-	-	-	-	C4	-	-	-	-	C2
14:00	-	-	-	C1	-	-	-	-	C5	-
15:00	-	-	C8	-	-	-	-	-	-	-

Instance 5 – Moderate



Weekly Timetable (Course IDs):									
Day	Friday	Monday	Thursday		Tuesday		Wednesday		
Room	R2	R4	R5	R2	R5	R1	R2	R1	R3
Time									
13:00	-	-	-	-	C10	-	-	-	-
14:00	-	-	C3	C9	-	-	-	C6	-
15:00	-	C4	-	-	-	-	-	-	C7
16:00	C8	-	-	-	-	C5	-	-	-
17:00	C1	-	-	-	-	-	C2	-	-

Instance 10 – Hard



Day	Friday		Monday			Thursday		Tuesday		Wednesday	
Room	R3	R5	R1	R2	R5	R2	R3	R3		R3	R4
Time											
10:00	-	-	-	-	C3	-	-	C15	-	-	-
11:00	-	-	C7	-	-	C6	-	C12	-	-	-
12:00	-	C11	-	C5	-	C4	-	-	-	-	-
13:00	-	C14	-	-	-	-	C13	-	-	-	C8
14:00	C9	-	-	-	-	-	-	-	-	C2	-
15:00	C10	-	-	-	-	-	-	-	-	-	C1

5. Key Observations:

- **GA without repair** handled easier instances well, showing high fitness and consistency.
- **Performance degraded** in harder instances due to increased constraint complexity.
- The **repair function** (when applied) improved feasibility rate and stability in test runs (though not included in main result table).
- Uniform crossover consistently outperformed single_point on average, though the latter sometimes found lower best fitness in hard instances.
- **Execution Time** remained acceptable even on the hardest instance (< 3 seconds avg).

6. Conclusion:

This study focused on solving the University Course Timetabling Problem using a Genetic Algorithm enhanced with domain-specific repair logic. Key contributions:

- Implemented a **flexible GA framework** with configurable crossover, mutation, and repair.
- Developed a **repair function** leveraging UCTP domain knowledge to fix violations.
- Conducted **systematic parameter tuning** across multiple instances and parameter values.
- Ran extensive experiments across **10 diverse instances**, showing performance trends by difficulty level.

The approach demonstrated that:

- **GA is effective** for medium-sized timetabling problems.
- **Repair functions** significantly improve solution feasibility and fitness.
- Proper tuning of hyperparameters like **population size, mutation rate, and crossover operator** is **critical** for performance.

For future work:

- Hybridize GA with local search (e.g., hill climbing).
- Incorporate **multi-objective optimization** (fairness, room preferences, etc.).
- Apply to real university data with custom constraints.